Microservices in Node

11 min read Updated 24 Jul 2026 Topic 69 of 70

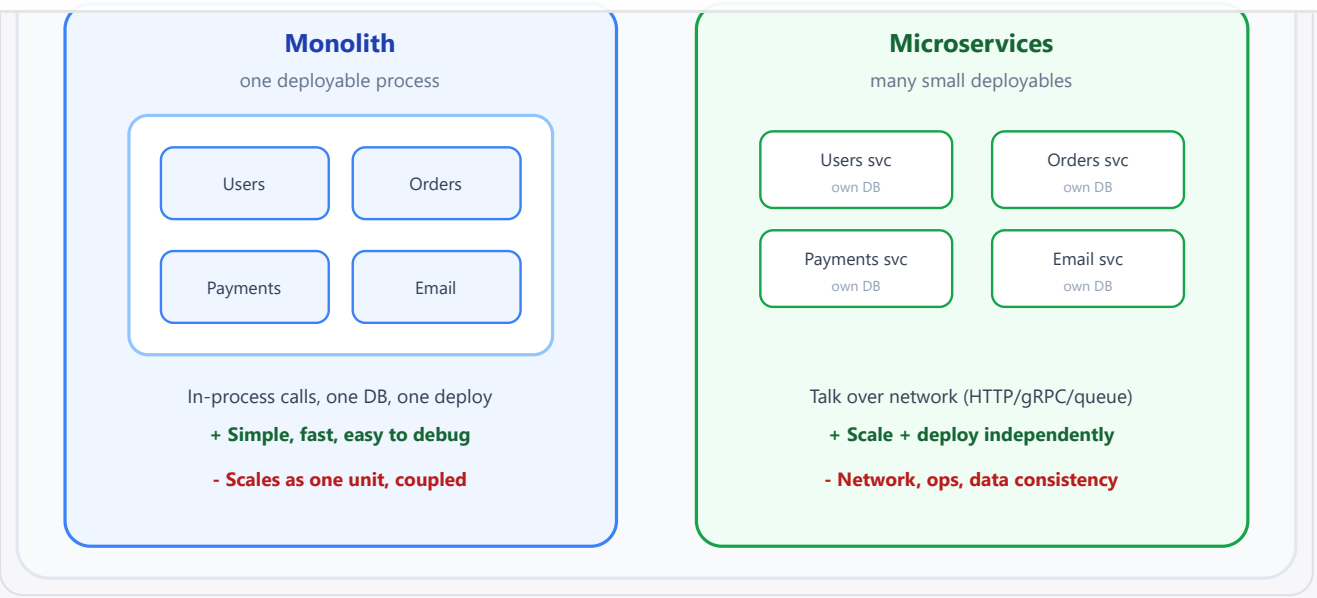
Why it matters

Every backend starts as a **monolith**: one codebase, one deploy, one database. That is not an embarrassment — it is the correct default, and it stays correct for a very long time. But as an organisation grows, a single deployable can become a bottleneck: fifty engineers stepping on one release pipeline, one memory leak taking the whole product down, and a checkout path that needs ten servers dragging along an admin panel that needs one.

Microservices split that one process into many small, independently deployable services, each owning a slice of the domain. It is one of the most over-applied ideas in the industry — teams adopt it for résumé reasons and inherit a distributed-systems tax they were not ready to pay. So the interview value here is almost entirely about *judgement*: recognising the real forces that justify the split, drawing service boundaries that do not leak, and naming honestly what you give up. In Node specifically, the lightweight process model makes spinning up services easy, which makes the discipline of *not* over-splitting even more important.

The concept, simply

A monolith keeps all its modules in one process; they call each other as ordinary function calls and share one database. Microservices put each capability in its own process, on its own schedule, usually with its own database, and they talk over the **network** instead of via function calls. The unit of independence is the key idea: you can deploy, scale, and (in a failure) lose one service without touching the others.



A monolith calls in-process over one DB — simple and fast; microservices trade that simplicity for independent deploy and scaling, paying with network calls, distributed data, and operational overhead.

The single most important concept is the **service boundary**. A good boundary follows a *business capability* that changes as a unit and owns its data — "Orders", "Payments", "Notifications" — not a technical layer like "the database service". The litmus test: two services should be able to evolve and deploy independently. If every change to Orders forces a coordinated change to Inventory, you have not drawn a boundary; you have drawn a fault line.

How it works (deep)

Data ownership is the rule that makes microservices actually independent: each service owns its data and no other service touches that database directly — the "database-per-service" pattern. The instant two services share a table, they are coupled at the schema and can no longer deploy independently; you have built a "distributed monolith", which has all the network cost of microservices and none of the independence. Cross-service data is obtained by *asking the owning service*, never by reaching into its store.

Communication comes in two flavours. *Synchronous* request/response (HTTP/REST or gRPC) is simple and immediate but creates temporal coupling — if Payments is down, the caller waiting on it is affected, and chains of synchronous calls multiply latency and failure probability. *Asynchronous* messaging (events over a queue or broker) decouples in time: Orders emits "OrderPlaced" and moves on; Notifications and Analytics react

answer is needed now, events for everything that can be fire-and-forget.

Because you cannot span a database transaction across services, **consistency becomes eventual** and multi-step workflows use the **Saga** pattern: a sequence of local transactions, each emitting an event that triggers the next, with *compensating actions* to undo prior steps if a later one fails (cancel the order if payment fails, refund the payment if shipping fails). This is a deliberate trade — you give up the simplicity of one ACID transaction to gain independence.

A cluster of supporting pieces make it workable: an **API gateway** as the single front door (auth, routing, rate limiting) so clients do not talk to dozens of services; **service discovery** so services find each other's moving addresses; **resilience patterns** — timeouts, retries with backoff and jitter, and **circuit breakers** that stop hammering a failing dependency — because in a distributed system partial failure is the normal case, not the exception; and **distributed tracing** with correlation IDs so one user action can be followed across ten services when it goes wrong. None of these exist in a monolith because in-process calls do not need them; every one of them is part of the tax.

Code examples

Two tiny Node services talking over HTTP. Note the *timeout* — a synchronous caller must never wait forever on a dependency.

JAVASCRIPT

Copy

```
// orders-service: needs user data it does not own → ask the users servic
import express from 'express';
const app = express();

async function getUser(id) {
  const ac = new AbortController();
  const t = setTimeout(() => ac.abort(), 800); // fail fast, do not hang
  try {
    const res = await fetch('http://users-svc:4001/users/' + id, { signal:
    if (!res.ok) throw new Error('users-svc ' + res.status);
    return await res.json();
  } finally {
```

```
clearTimeout(t);
}

app.get('/orders/:id', async (req, res) => {
  try {
    const order = { id: req.params.id, userId: '1', total: 999 };
    const user = await getUser(order.userId); // cross-service call
    res.json({ ...order, user });
  } catch (e) {
    res.status(503).json({ error: 'user lookup failed' });
  }
});

app.listen(4002);
```

A minimal circuit breaker — after repeated failures it "opens" and fails fast instead of piling requests onto a dead dependency, then probes to recover.

JAVASCRIPT Copy

```
class CircuitBreaker {
  constructor(fn, { threshold = 5, cooldownMs = 10_000 } = {}) {
    this.fn = fn; this.threshold = threshold; this.cooldownMs = cooldownMs
    this.failures = 0; this.state = 'closed'; this.openedAt = 0;
  }
  async call(...args) {
    if (this.state === 'open') {
      if (Date.now() - this.openedAt < this.cooldownMs) throw new Error('c
      this.state = 'half-open'; // allow one probe
    }
    try {
      const out = await this.fn(...args);
      this.failures = 0; this.state = 'closed';
      return out;
    } catch (err) {
      if (++this.failures >= this.threshold) {
        this.state = 'open'; this.openedAt = Date.now();
      }
      throw err;
    }
  }
```

Event-driven communication instead of a synchronous call — Orders emits an event and returns immediately; other services react on their own time. (Broker details are the next topic.)

JAVASCRIPT Copy

```
// orders-service publishes; it does NOT wait for email/analytics
import amqplib from 'amqplib';

const conn = await amqplib.connect(process.env.AMQP_URL);
const ch = await conn.createChannel();
await ch.assertExchange('domain-events', 'topic', { durable: true });

function publish(routingKey, payload) {
  ch.publish('domain-events', routingKey,
    Buffer.from(JSON.stringify(payload)),
    { persistent: true, contentType: 'application/json' });
}

// after saving the order locally:
publish('order.placed', { orderId: 'o-42', userId: '1', total: 999 });
// respond to the client now; notifications + analytics consume asynchrono
```

A health/readiness endpoint — orchestrators (Kubernetes) use these to route traffic only to healthy instances, a basic requirement once you run many services.

JAVASCRIPT Copy

```
import express from 'express';
const app = express();

let ready = false;
app.get('/livez', (_req, res) => res.send('ok'));       // process is up
app.get('/readyz', (_req, res) =>                          // deps are usa
  ready ? res.send('ready') : res.status(503).send('not ready'));
```

```
  await connectDbAndBroker(); // whatever this service depends on
  ready = true;
})();

app.listen(4001);
async function connectDbAndBroker() {}
```

Common mistakes & gotchas

- **Splitting too early.** Microservices solve *organisational* and *scaling* problems you do not have on day one. A "monolith-first" approach lets boundaries reveal themselves before you cement them into network calls.
- **Sharing a database between services.** The fastest way to build a distributed monolith: now two services are coupled at the schema and must deploy together, so you pay the network cost and get none of the independence.
- **Drawing boundaries by technical layer** ("a data service", "a logic service") instead of by business capability. Capability boundaries change as a unit; layer boundaries force every feature to cut across many services.
- **Assuming the network is reliable.** Calls time out, retry, and arrive twice. Without timeouts, retries-with-backoff, idempotency, and circuit breakers, one slow dependency cascades into a full outage.
- **Chatty synchronous chains.** Service A calls B calls C calls D to serve one request — latency and failure probability multiply. Prefer events, or aggregate at a gateway, over deep synchronous fan-out.
- **Expecting ACID transactions across services.** There is no distributed transaction to lean on; multi-step consistency needs Sagas with compensating actions and idempotent handlers.
- **No distributed tracing.** When a request touches ten services, a stack trace in one of them tells you almost nothing. Without correlation IDs and tracing, debugging is guesswork.
- **Ignoring the operational cost.** Every service multiplies CI/CD, monitoring, on-call, and deployment surface. Ten services means ten of everything.

- **Start monolith-first;** extract a service only when a real force (independent scaling, team autonomy, isolation of a failure domain) demands it — modular boundaries inside the monolith make later extraction cheap.
- **One service owns its data;** never share a database. Cross-service data comes from the owning service's API or from events, never a direct table read.
- **Prefer async events for anything fire-and-forget;** reserve synchronous calls for when you truly need an answer now, and always bound them with timeouts.
- **Make every consumer idempotent** (dedupe on a message/operation id) because at-least-once delivery and retries mean messages will repeat.
- **Build in resilience:** timeouts, retry-with-backoff-and-jitter, circuit breakers, and sensible fallbacks — treat partial failure as normal.
- **Put a gateway out front** for auth, routing, and rate limiting, and standardise cross-cutting concerns (logging, tracing, correlation IDs) across services.
- **Version contracts and evolve them additively;** a breaking change to a shared event or API must not require a lockstep deploy of every consumer.
- **Invest in observability first:** centralised logs, metrics, and distributed tracing are prerequisites, not afterthoughts, once you cross a couple of services.

⭐ Interview lens

Q: When would you choose microservices over a monolith? When concrete forces justify the tax: independent scaling of hot paths, autonomy for many teams shipping in parallel, isolating a failure or security domain, or differing technology needs. Absent those, a modular monolith is simpler, faster, and easier to operate. *Trap:* "because it scales better" — a well-built monolith scales horizontally too; microservices scale *parts* independently, which is a different claim.

Q: How do you decide service boundaries? Around business capabilities that change and deploy as a unit and own their data (Orders, Payments, Notifications)

— cut around technical layers. The test is independent evolvability: if changing one always forces changing another, the boundary is wrong.

Q: What is a "distributed monolith" and why is it the worst case? Services that must be deployed together — usually because they share a database or have tightly coupled synchronous contracts. You pay the full cost of the network and distributed operations while keeping the coupling of a monolith, getting the downsides of both.

Q: You cannot do a transaction across services. How do you keep data consistent? Accept eventual consistency and use the Saga pattern: a chain of local transactions linked by events, with compensating actions to roll back completed steps when a later one fails. Handlers must be idempotent because messages can be redelivered.

Q: Service B is slow. How do you stop it from taking down A? Timeouts so A never waits indefinitely, retries with backoff and jitter for transient blips, and a circuit breaker that trips after repeated failures to fail fast (with a fallback) instead of queuing doomed requests. Bulkheads isolate the resource pool so one dependency cannot starve the rest.

Q: Sync REST/gRPC vs async messaging between services — how do you choose? Synchronous when the caller genuinely needs the result to proceed (a read it will render now) — accept the temporal coupling and bound it. Asynchronous events when the work can happen independently (notifications, analytics, downstream updates) — you gain decoupling and resilience but must design for eventual consistency and duplicate delivery.

Cheat-card

- **Monolith** = one deploy, in-process calls, one DB — the right default. **Microservices** = many deploys, network calls, DB-per-service.
- **Boundary rule:** by business capability that owns its data and deploys independently — never by technical layer.
- **Never share a database →** sharing one builds a "distributed monolith" (all cost, no independence).
- **Comms:** sync (HTTP/gRPC) when you need an answer now; async events for fire-and-forget resilience.
- **No cross-service ACID →** Sagas + compensating actions + idempotent, at-least-once handlers.
- **Resilience kit:** timeouts, retry+backoff+jitter, circuit breakers, bulkheads; assume partial failure.
- **Ops tax:** gateway, service discovery, distributed tracing + correlation IDs, N× CI/CD and monitoring.
- **Default advice:** monolith-first; extract a service only when a real force demands it.

QUICK CHECK — no marks, just for you.

1. How should you draw service boundaries in a microservices architecture?

- ○ Around business capabilities that change and deploy as a unit and own their data (Orders, Payments) — not around technical layers
- ○ Around technical layers such as "a data service" and "a logic service"
- ○ One service per database table for maximum granularity
- ○ Around individual developers so each owns one service

Check answer

2. What is a "distributed monolith" and why is it the worst case?

- A single service that grew too large to deploy
- Services that must be deployed together — usually because they share a database or tightly coupled sync contracts — so you pay the network cost while keeping monolith coupling
- A message queue shared across too many consumers

Check answer

3. You cannot run one ACID transaction across services. How do you keep a multi-step workflow consistent?

- Use a global lock across all service databases
- Route every write through a single shared database to regain ACID
- Accept that consistency is impossible and drop failed steps
- Accept eventual consistency and use the Saga pattern: a chain of local transactions linked by events, with compensating actions to undo prior steps and idempotent handlers

Check answer

‹ Previous
GraphQL basics

Next ›
Message queues & async jobs

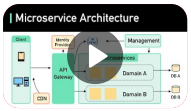
LEARN MORE

ARTICLES & DOCS

Microservices (Martin Fowler & James Lewis)
martinfowler.com

Pattern: Microservice Architecture
microservices.io

VIDEOS



What Are Microservices Really All About? (And When Not To Use It)
ByteByteGo

Hand-picked references for this lesson.